
7_Test_Environment_Setup (Detailed Explanation)

□ Purpose of this Phase

This phase ensures that the **test environment is properly prepared before execution** of Selenium automation scripts for the SauceDemo login application.

In simple terms:

“We make sure everything needed to run automation tests is installed, configured, and working.”

Application Under Test (AUT)

- **Application:** SauceDemo Login Module
- **URL:** https://www.saucedemo.com
- **Type:** Web-based E-commerce demo application
- **Purpose:** Used to validate login functionality using multiple test data sets (CSV-driven testing)

System Configuration

✓ Operating System

- Windows 10 / Windows 11

✓ Browser Support

- Google Chrome (Primary)
- Microsoft Edge (Optional)

✓ Browser Driver

- ChromeDriver (matching browser version)

Software Requirements

✓ Java

- Java JDK 17 / 21 (recommended for Selenium + TestNG stability)

✓ IDE

- Eclipse IDE / IntelliJ IDEA

✓ Build Tool

- Maven (for dependency management)

Required Dependencies (Maven)

In `pom.xml`, following libraries are configured:

- Selenium WebDriver
- TestNG
- Apache POI (if needed for Excel handling)
- Extent Reports (for reporting)
- OpenCSV (for CSV data-driven testing)

Project Configuration Setup

config.properties file contains:

- Application URL
- Browser name (Chrome)
- Implicit wait time
- Screenshot path
- Report path

Example:

```
url=https://www.saucedemo.com
browser=chrome
timeout=10
```

□ Test Data Setup

CSV File:

- File: `LoginData.csv`

- Location: `/src/test/resources/`
- Contains:
  - Username
  - Password
  - Expected Result

This supports **Data Driven Testing (DDT)**.

Framework Structure Setup

✓ Page Object Model (POM)

- `LoginPage.java`
  - Locators for username, password, login button
  - Methods like `enterUsername()`, `clickLogin()`

✓ Base Class Setup

- `BaseClass.java`
  - Browser initialization
  - Driver setup
  - Common reusable methods

✓ Utilities Setup

- `CSVReaderUtil.java` → reads test data from CSV
- `ConfigReader.java` → reads config.properties
- `ScreenshotUtil.java` → captures failure screenshots
- `ExtentManager.java` → handles report generation

Execution Setup

✓ testng.xml

- Controls test execution flow
- Defines:
  - Test classes
  - Parallel execution (if needed)

Screenshot Setup

- Pass screenshots → `/Screenshots/Pass`
- Fail screenshots → `/Screenshots/Fail`

Captured automatically using `ScreenshotUtil`

Reporting Setup

- Extent Reports configured via:
  - `extent-config.xml`
- Generates:
  - HTML report after execution
